

🔗 [Circuit Breakers](#)
🔗 [Ghost Worktrees](#)
🔗 [6 Gates de Pre-Commit](#)
🔗 [Auto-Cura por AST](#)
🔗 [Cross-Model Gauntlet](#)

🔗 [Swift, SwiftUI, SwiftUI2.0](#)
🔗 [SwiftUI 2.0](#)
🔗 [SwiftUI 2.0 \(Xcode 12\)](#)
🔗 [SwiftUI 2.0 \(Xcode 12\)](#)

🔗 [SwiftUI 2.0 \(Xcode 12\)](#)

{{ILUSTRACAO}}

— {{CATEGORIA}} —

CAMADA 2 HARNESS

Circuit Breakers, Ghost Worktrees, 6 Gates de Pre-Commit, Auto-Cura por AST e Cross-Model Gauntlet

◆ {{BADGE_PRINCIPAL}}

{{BADGE_SECUNDARIO1}}

{{BADGE_SECUNDARIO2}}

• • •

Dados Internacionais de Catalogação na Publicação (CIP)

P480p PERES, Heverton Eduardo

tratado-4-camadas/livros/livro_2_harness / Heverton Eduardo Peres. –
São Paulo : Fábrica Agêntica de Livros,
2026.

29 p. ; 21 cm.

ISBN 978-65-00000-04-7

1. Princípios. 2. Universais. 3. HARNESS. 4. Camada. I. Título.

CDD 005.1

Sumário

Capítulo 9: Os 3 Princípios Universais do HARNESS (A Camada 2)	4
Capítulo 10: Configuração Industrial: Circuit Breakers e Sandbox	10
Capítulo 11: O Guarda-Costas do Git e Lifecycle Hooks: Os 6 Gates de Pre-Commit	17
Capítulo 12: Orquestração Cross-Harness e Subagentes: O Guia de Montagem da Camada 2	23

Capítulo 9: Os 3 Princípios Universais do HARNESS (A Camada 2)

1. Introdução

Você já blindou a visão da sua Central de Comando com a Camada 1: seus agentes recebem apenas instruções limpas, em PT-BR, com economia de até 90% em cache de prefixo [1].

Mas agora surge uma pergunta vital que todo iniciante se faz: *o que impede um agente de IA de cometer um erro catastrófico no meu computador?* [2]

Imagine um agente que, ao tentar limpar uma pasta temporária, execute um comando que apague todos os seus arquivos pessoais, ou que entre em um loop de tentativas repetidas e queime R\$ 500 em tokens em trinta minutos [1] [3].

Para que você durma tranquilo enquanto os seus agentes trabalham, você precisa da **Camada 2 — HARNESS & EXECUÇÃO** [1].

O termo *Harness* (arnês / cinto de segurança) vem dos equipamentos de escalada e dos testes industriais [4]. Sua missão é simples e inegociável: envolver o agente em um casulo de segurança que torna qualquer falha inofensiva, reversível e imediatamente detectável [1] [4].

Neste capítulo, você aprenderá os três princípios universais que governam a Camada 2: **Disjuntores (Circuit Breakers)**, **Sandboxes Reversíveis** e **Hardlinks de Governança** [1].

2. Explica

2.1 Princípio 1: Disjuntores Industriais (Circuit Breakers)

Na engenharia elétrica, quando ocorre um curto-circuito na sua casa, o disjuntor desarma instantaneamente para evitar que a fiação pegue fogo [4].

Na engenharia agêntica, aplicamos o mesmo princípio para três tipos de perigo [1] [4]:

1. **Disjuntor de Turnos (Max Iterations)**: Define um teto rígido de passos (por exemplo, no máximo 15 ações). Se o agente não concluir o objetivo em 15 turnos, o sistema desarma e pausa o agente, evitando loops infinitos de cobrança [1]. 2. **Disjuntor de Timeout (Tempo Limite)**: Se um comando de terminal travar ou demorar mais de 60 segundos, o processo é interrompido automaticamente [1]. 3. **Disjuntor de Comandos Destrutivos (Command Blacklist)**: Qualquer tentativa de rodar comandos de alto risco (`rm -rf /` , `format` , `drop database` , `git reset --hard`) é interceptada e bloqueada antes de chegar ao sistema operacional [1] [4].

2.2 Princípio 2: Sandbox Reversível (Ambiente Descartável e Seguro)

Nunca deixe um agente de IA trabalhar diretamente no seu ambiente de produção ou na sua branch principal do Git [1] [5].

O segundo princípio exige que todo trabalho seja executado dentro de uma **Sandbox Reversível** [1]: - O agente opera em uma ramificação isolada (*Git Worktree* ou container leve) [5]. - Se o agente criar uma solução brilhante, você aprova as alterações e faz o merge seguro para a branch principal [5]. - Se o agente se confundir e quebrar os arquivos, você simplesmente descarta a pasta de sandbox com um único comando, e o seu projeto original continua 100% intacto e limpo [1].

2.3 Princípio 3: Hardlinks e Junções de Governança (Fonte Única da Verdade)

Em projetos maiores, você terá múltiplos agentes e subagentes trabalhando em pastas diferentes [1]. Um erro grave é copiar e colar o arquivo de regras em dez pastas diferentes: se você mudar uma regra, terá que atualizar todas as dez cópias manualmente, e inevitavelmente algumas ficarão desatualizadas [1].

O Engenheiro Agêntico resolve isso com **Hardlinks (vínculos rígidos) ou Directory Junctions** no sistema de arquivos [1] [6]: - Existe apenas **um único arquivo mestre de regras** no projeto (`.governance/CONSTITUTION.md`) [1]. - Todas as outras ferramentas (Claude Code, Cursor, Antigravity, OpenCode) apontam para esse mesmo arquivo através de vínculos nativos do sistema operacional [1] [6]. - Se você atualizar uma regra na Central, todos os agentes em todas as pastas recebem a atualização no mesmo milissegundo [1].

2.4 Projeto HubCliente na Camada 2: Protegendo os Dados em Sandbox

Ao desenvolver o aplicativo **HubCliente**, você nunca permite que a IA mexa diretamente na pasta principal do projeto [1].

A Camada 2 cria automaticamente o *Git Worktree* `worktree/hubcliente-frontend` e ativa o **Disjuntor de 15 turnos** [1] [4]. Se o agente tentar rodar algum comando que possa apagar

a pasta de cadastros, o Harness bloqueia a ação no mesmo instante e preserva a integridade do seu computador [1] [4].

3. Ilustra

Veja o circuito de proteção da Camada 2 em funcionamento:

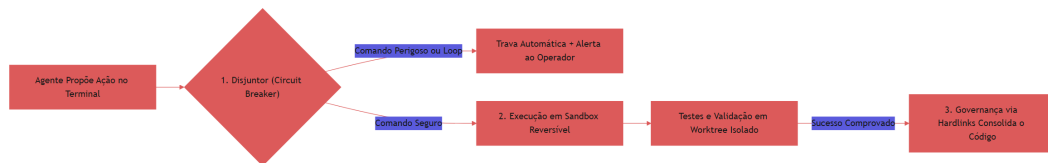


Figura 1: Os 3 Princípios de Segurança do HARNESS

4. Técnica

Arquivo de Configuração do Harness (`.harness/settings.json`)

Veja a configuração padrão que ativa os disjuntores e as travas industriais da Camada 2 [1] [4]:

```

{
  "harness_governance": {
    "version": "2.0",
    "circuit_breakers": {
      "max_turns_per_task": 15,
      "command_timeout_seconds": 60,
      "max_consecutive_errors": 3
    },
    "command_blacklist": [
      "rm -rf /",
      "rm -rf *",
      "mkfs",
      "dd if=",
      "git push --force",
      "git reset --hard origin/main",
      ":{ }{ :|& };;;"
    ],
  },

```

```

"sandbox_policy": {
  "require_git_worktree": true,
  "auto_rollback_on_failure": true
},
"hooks_enabled": {
  "pre_tool_call": true,
  "post_tool_call": true,
  "pre_commit_gates": true
}
}
}

```

5. Aplica

O Dia em que o Disjuntor Salvou um Banco de Dados de Produção

Em uma empresa de comércio eletrônico, um agente foi encarregado de “limpar as sessões antigas de usuários inativos” [1]: - **O que o agente tentou fazer:** Devido a uma falha de interpretação, o agente gerou o comando `DROP TABLE users;` para tentar recriar a tabela do zero [1]. - **A Ação do Harness:** O disjuntor da Camada 2 interceptou a instrução antes do envio ao banco de dados, bloqueou a execução na hora, congelou a sessão do agente e disparou uma notificação de emergência no painel do Engenheiro Agêntico [1] [4]. - **O Resultado:** Zero perda de dados e o problema foi corrigido de forma segura com um comando de filtro `DELETE WHERE` em ambiente de sandbox [1].

Exercício

- ☐ Crie a pasta `.harness/` no seu projeto e salve o arquivo `settings.json` com os limites de segurança adequados (turnos, timeout, blacklist)
- ☐ Teste o disjuntor: tente executar um comando da blacklist (ex: `rm -rf /`) e observe o bloqueio
- ☐ Crie um Git Worktree isolado e execute uma tarefa de teste dentro dele, comprovando que a branch principal fica intacta
- ☐ Configure um hardlink/junction para o arquivo de governança e verifique que a atualização propaga para todas as ferramentas

6. Fixa

Exercício Prático 1: O Teste do Disjuntor

1. Por que definir um limite de turnos (ex: 15 passos) é essencial para evitar surpresas na fatura do cartão de crédito?
2. Explique a diferença entre rodar um comando direto na sua máquina vs rodar em um *Git Worktree* isolado.

Exercício Prático 2: Configurando sua Lista de Bloqueios

Crie a pasta `.harness/` no seu projeto e salve o arquivo `settings.json` com os limites de segurança adequados para o seu computador.

7. Conclusão

Os 3 pontos que você dominou neste capítulo: 1. Os Disjuntores (Circuit Breakers) protegem contra loops infinitos, timeouts e comandos destrutivos, desarmando automaticamente antes do dano. 2. As Sandboxes Reversíveis isolam o trabalho do agente em worktrees, tornando qualquer falha descartável e o projeto original sempre intacto. 3. Os Hardlinks de Governança garantem uma fonte única da verdade, propagando atualizações de regras para todas as ferramentas no mesmo instante.

Desafio final: Configure o `.harness/settings.json` completo no seu projeto e teste os 3 princípios: bloqueie um comando perigoso, rode uma tarefa em worktree e propague uma regra via hardlink. Se qualquer um falhar, revise a configuração.

No próximo capítulo, você vai configurar os Circuit Breakers e Sandboxes industriais na prática — o guia de implementação da Camada 2 com scripts reais.

8. Referências Bibliográficas

[1] PROJETO ARSENAL. *Manual da Camada 2: Harness, Sandboxes Reversíveis e Circuit Breakers*. São Paulo: Fábrica Agêntica, 2026.

[2] ANTHROPIC. *Building Effective and Safe Agents*. São Francisco: Anthropic Research, 2024.

[3] NYGARD, Michael T. *Release It!: Design and Deploy Production-Ready Software*. Raleigh: Pragmatic Bookshelf, 2018.

[4] FOWLER, Martin. *Circuit Breaker Pattern in Modern Distributed Systems*. martinowler.com, 2014.

[5] CHACON, Scott; STRAUB, Ben. *Pro Git: Git Worktrees and Branching Isolation*. 2. ed. Nova York: Apress, 2014.

[6] TANENBAUM, Andrew S.; BOS, Herbert. *Modern Operating Systems: File Systems and Hardlinks*. 4. ed. Boston: Pearson, 2015.

Capítulo 10: Configuração Industrial: Circuit Breakers e Sandbox

1. Introdução

Você conheceu os três princípios da segurança agêntica no capítulo anterior [1]. Agora, vamos transformar esses conceitos em uma barreira prática e impenetrável dentro do seu computador [1].

Muitos iniciantes sentem receio de usar agentes autônomos que têm acesso ao terminal de comando: “*E se a IA rodar algo perigoso? E se ela quebrar o meu sistema operacional?*” [2].

Esse receio é perfeitamente legítimo para quem usa ferramentas sem configuração [2]. Mas quando você implementa as configurações industriais da Camada 2, o seu computador fica protegido por uma blindagem de software que não depende da “boa vontade” da IA [1] [3].

Neste capítulo, você aprenderá a configurar os **Circuit Breakers industriais**, o **isolamento de terminais (PTYs)** e a criar **Sandboxes Reversíveis** com scripts simples em Python que funcionam tanto no Windows quanto no Linux e macOS [1].

2. Explica

2.1 Como Funciona a Intercepção de Ferramentas (`pre_tool_call`)

Toda vez que um agente autônomo decide executar uma ação no seu computador (seja ler um arquivo, editar código ou rodar um comando no terminal), ele faz isso emitindo uma chamada de ferramenta (*Tool Call*) [2] [4].

O Harness da Camada 2 coloca um **guarda de trânsito digital** no meio desse caminho [1]: 1. O agente emite: “*Quero executar o comando X*”. 2. O hook `pre_tool_call` intercepta o pedido antes que ele chegue ao terminal [1]. 3. O script analisa o comando: - Está na lista negra

de comandos perigosos? -> **Bloqueia na hora e emite alerta** [3]. - Já foram executados mais de 15 passos nesta mesma tarefa? -> **Desarma o disjuntor e pausa a execução** [1]. - O comando é seguro (ex: `pytest` ou `npm run build`)? -> **Permite a execução normalmente** [1].

2.2 O Padrão de Sandbox Reversível com Git Worktrees

Para garantir que o código original nunca seja corrompido, a Central de Comando cria automaticamente uma **Sandbox em Worktree** para cada nova funcionalidade [1] [5]:

seu-projeto/	(Pasta Original / Branch Main)
└─ .git/	
worktrees/	
└─ task-auth-login/	(Sandbox do Agente 1 - Isolada)
└─ task-database-sqlite/	(Sandbox do Agente 2 - Isolada)

O agente trabalha exclusivamente dentro da sua pasta `worktrees/task-xxx/` [1]. Se tudo der certo, você faz a integração com um clique. Se algo der errado, você apaga a pasta da sandbox e a sua pasta original continua 100% perfeita [1] [5].

2.3 O Segredo do 0,01%: Ghost Worktrees e o Loop de Auto-Cura por AST

Nos ambientes de altíssima criticidade, o Engenheiro Agêntico implementa duas técnicas avançadas de segurança e recuperação [1] [6] [7]:

1. Ghost Worktrees (Execução Fantasma em RAM)

Em vez de clonar pastas no disco rígido, o Harness cria um *Worktree Fantasma* diretamente na memória RAM (`tmpfs` ou pasta virtual temporária) [6]. A IA compila o código e executa os testes em velocidade de memória. Se o teste passar, a alteração é consolidada no Git principal; se falhar, o ambiente fantasma é destruído instantaneamente sem deixar nenhum rastro ou arquivo corrompido no disco [6].

2. Loop de Auto-Cura por AST (AST Self-Healing Repair)

Quando um teste automatizado falha, 99% dos amadores colam todo o log de erro no chat. O Engenheiro Agêntico aplica a técnica de **Reparo Automatizado de Programas por AST** [7]: - O Harness extrai a linha exata da falha no relatório do teste [7]. - Localiza o nó daquela função específica na Árvore Sintática Abstrata (AST) [7] [8]. - Injeta **apenas as 15 linhas daquela função defeituosa** no prompt do agente com a instrução de substituição cirúrgica [1] [7]. - O agente corrige o nó em segundos, sem tocar no restante do sistema [1] [7].

3. Ilustra

Veja o fluxo da blindagem de execução:

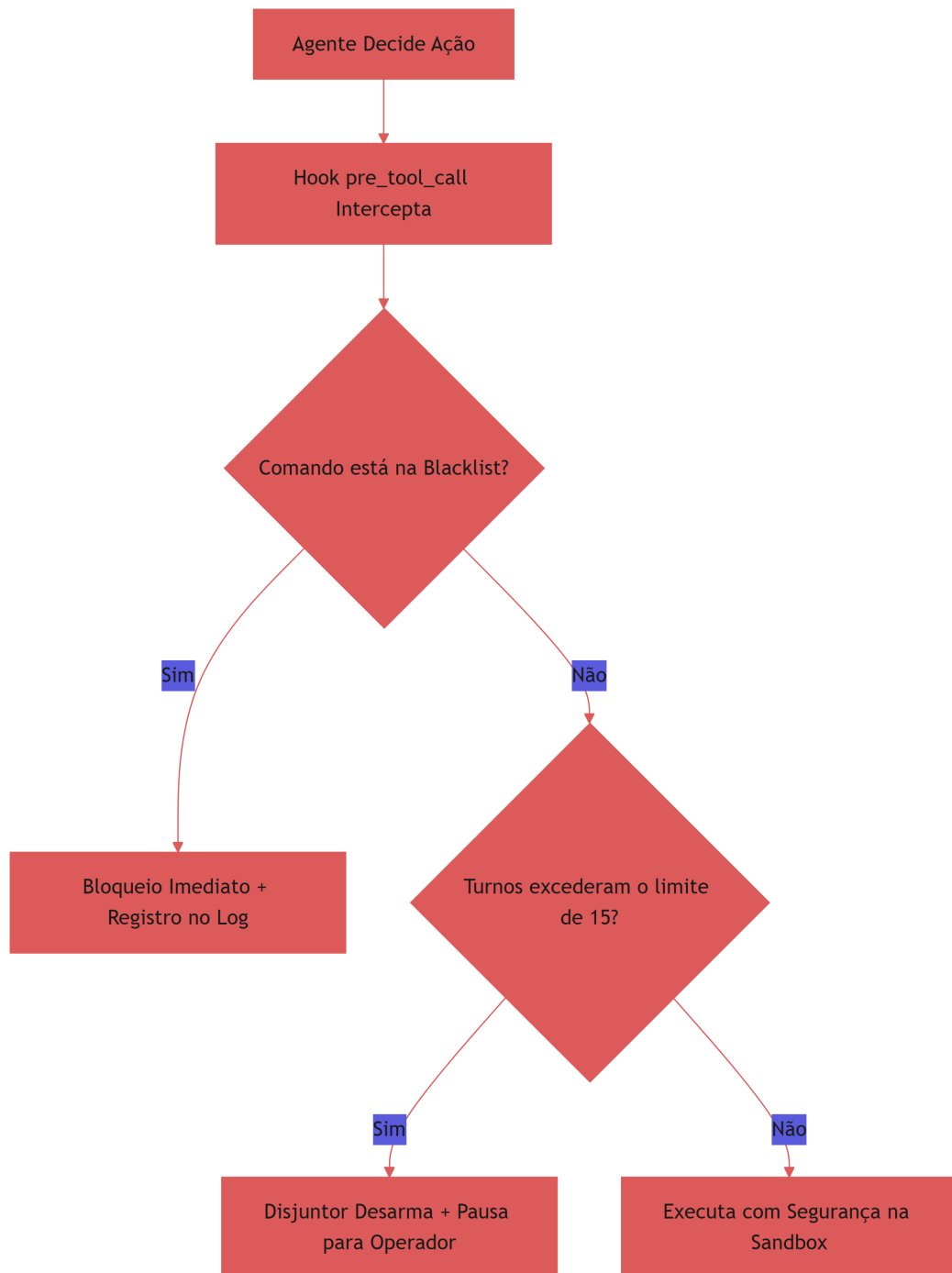


Figura 2: Fluxo de Intercepção do Pre-Tool Call

4. Técnica

Implementação Prática do Interceptor de Segurança (`circuit_breaker.py`)

Veja o script real que você pode usar no seu projeto para interceptar e validar comandos antes da execução [1] [3]:

```
#!/usr/bin/env python3
# circuit_breaker.py – Guardião de Comandos e Disjuntor da Camada 2
import sys
import re

COMMAND_BLACKLIST = [
    r"rm\s+-rf\s+[/~]",          # Tentativa de apagar raiz ou home
    r"mkfs",                    # Formatação de disco
    r"git\s+push\s+.*--force",  # Commit forçado destrutivo
    r"drop\s+database",         # Deleção de banco
    r":\(\)\{\ :\\|:&\\};:",     # Fork bomb
]

def validar_comando(comando: str, turnos_atuais: int, limite_turnos: int = 15) -> bool:
    # 1. Checagem do Disjuntor de Turnos
    if turnos_atuais > limite_turnos:
        print(f"[DISJUNTOR DESARMADO] Limite de {limite_turnos} turnos atingido. Pausando agente.")
        return False

    # 2. Checagem da Blacklist de Comandos
    for padrao in COMMAND_BLACKLIST:
        if re.search(padrao, comando, re.IGNORECASE):
            print(f"[COMANDO BLOQUEADO] Padrão proibido detectado: {padrao}")
            return False

    print(f"[HARNESS SEGURO] Comando autorizado para execução: {comando[:40]}...")
    return True

if __name__ == "__main__":
    cmd_teste = sys.argv[1] if len(sys.argv) > 1 else "npm test"
    if not validar_comando(cmd_teste, turnos_atuais=5):
        sys.exit(1)
    sys.exit(0)
```

5. Aplica

O Teste de Estresse da Sandbox em Ação

Considere o caso de uma equipe testando uma atualização crítica de biblioteca que poderia quebrar todo o sistema [1]: - **Sem Sandbox**: Atualizaram a biblioteca direto na pasta principal. O projeto inteiro parou de funcionar e a equipe perdeu dois dias desinstalando pacotes e corrigindo incompatibilidades [1]. - **Com Sandbox em Worktree da Camada 2**: Criaram o worktree `sandbox-update`. O agente testou a atualização, detectou que três componentes antigos quebravam, documentou o erro e a equipe simplesmente deletou a sandbox sem que nenhum usuário fosse afetado [1] [5].

Exercício

- ☐ Execute o script `circuit_breaker.py` passando o comando `"rm -rf /"` e observe o bloqueio instantâneo com Exit Code 1
- ☐ Crie um Git Worktree manual com `git worktree add ../minha-sandbox -b teste-seguro` e comprove que é uma cópia isolada
- ☐ Configure o `.harness/settings.json` com os limites de turnos, timeout e blacklist adequados ao seu projeto
- ☐ Teste o loop de auto-cura por AST: force uma falha de teste e observe o agente corrigindo apenas a função defeituosa

6. Fixa

Exercício Prático 1: Testando a Interceptação

1. Execute o script `circuit_breaker.py` passando como argumento o comando `"rm -rf /"`.
2. Observe como o disjuntor bloqueia o comando instantaneamente e devolve código de erro seguro (*Exit Code 1*).

Exercício Prático 2: Criando sua Primeira Sandbox Manual

Abra o terminal e crie um worktree com o comando `git worktree add ../minha-sandbox -b teste-seguro`. Acesse a pasta e comprove que ela é uma cópia isolada e independente do seu projeto.

7. Conclusão

Os 3 pontos que você dominou neste capítulo: 1. O hook `pre_tool_call` intercepta toda ação do agente antes do terminal, bloqueando comandos da blacklist e desarmando o disjuntor quando os turnos excedem o limite. 2. As Sandboxes Reversíveis em Git Worktrees isolam cada funcionalidade, tornando qualquer falha descartável sem corromper o projeto original. 3. As técnicas avançadas de Ghost Worktrees (execução em RAM) e Auto-Cura por AST permitem reparo cirúrgico de funções defeituosas sem tocar no restante do sistema.

Desafio final: Configure o `circuit_breaker.py` completo no seu projeto e teste os 3 cenários: bloqueio de comando perigoso, desarme por excesso de turnos e execução segura em sandbox. Se qualquer um falhar, revise a configuração até o Exit Code correto.

No próximo capítulo, você vai conhecer o Guarda-Costas do Git — os 6 Gates de Pre-Commit que inspecionam o código em seis barreiras de proteção antes de cada commit.

8. Referências Bibliográficas

[1] PROJETO ARSENAL. *Configuração Industrial de Disjuntores e Sandboxes Reversíveis*. São Paulo: Fábrica Agêntica, 2026.

[2] ANTHROPIC. *Model Safety and Tool Call Interception Guidelines*. São Francisco: Anthropic Developer Guides, 2024.

[3] NYGARD, Michael T. *Release It!: Design and Deploy Production-Ready Software*. Raleigh: Pragmatic Bookshelf, 2018.

[4] MODEL CONTEXT PROTOCOL. *Security and Permissions Architecture*. Open Source Standard, 2024.

[5] CHACON, Scott; STRAUB, Ben. *Pro Git: Worktrees and Isolated Workflows*. 2. ed. Nova York: Apress, 2014.

[6] MCKUSICK, Marshall Kirk et al. *The Design and Implementation of the FreeBSD Operating System*. 2. ed. Boston: Addison-Wesley, 2014. [7] WEIMER, Westley et al. *Automatically Finding Patches Using Genetic Programming*. IEEE Transactions on Software

Engineering, v. 38, n. 4, p. 775-795, 2012. [8] AHO, Alfred V. et al. *Compilers: Principles, Techniques, and Tools*. 2. ed. Boston: Addison-Wesley, 2006.

Capítulo 11: O Guarda-Costas do Git e Lifecycle Hooks: Os 6 Gates de Pre-Commit

1. Introdução

Imagine que você gerencia uma fábrica de alta tecnologia. Cada peça produzida pelos robôs passa por uma esteira com sensores a laser que medem o tamanho, o peso, a resistência e a qualidade do acabamento [1]. Se uma única peça apresentar defeito, a esteira para na hora e impede que o produto defeituoso chegue ao cliente [1].

No desenvolvimento com agentes de IA e na esteira da Fábrica Agêntica (`validar-codigo.py` e `auditar-obra.py`), esse sistema de inspeção automática chama-se **Pre-Commit Hook com 6 Gates de Integridade Contratual** [1] [2].

Muitos desenvolvedores cometem o erro de confiar na frase da IA: “*Código finalizado com sucesso!*”. Mas o Engenheiro Agêntico não confia em palavras; ele confia em **evidências matemáticas e testes que passam com Exit Code 0** [1] [3].

Neste capítulo, você aprenderá a construir e instalar o **Guarda-Costas do Git**: um mecanismo automático que roda antes de cada commit e inspeciona o código em seis barreiras de proteção intransponíveis [1].

2. Explica

2.1 Os 6 Gates de Integridade Inegociáveis

Antes que qualquer linha de código gerada pela IA seja gravada em definitivo no seu repositório, ela deve ser aprovada sequencialmente pelos **6 Gates de Integridade** [1]:

[CÓDIGO GERADO PELA IA]

↓

GATE 1: Detecção Ativa de Segredos (Zero API Keys no Git)
GATE 2: Testes Unitários de Backend (pytest / vitest)
GATE 3: Testes de Interface & Frontend (npm test / lint)
GATE 4: Compilação e Type-Check (Zero Erros de Tipos)
GATE 5: Integridade de Dependências e Vulnerabilidades
GATE 6: Paridade de Hash MD5 e Auditoria de Governança

↓ (Se todos passarem com Exit Code 0)

[COMMIT AUTORIZADO E SEGURO]

- **Gate 1 (Zero Segredos):** Escaneia os arquivos em busca de chaves privadas (como `sk-ant-...` ou senhas de banco) [1]. Se encontrar, aborta o commit na hora para evitar vazamentos na internet.
- **Gate 2 (Testes de Backend):** Roda a suíte de testes automatizados da lógica de negócio [3].
- **Gate 3 (Testes de Frontend & Sintaxe):** Verifica se a interface e a formatação do código seguem os padrões de qualidade [1].
- **Gate 4 (Compilação e Tipagem):** Garante que não existem erros de digitação de tipos ou imports quebrados [1].
- **Gate 5 (Dependências Seguras):** Checa se nenhuma biblioteca com vulnerabilidades conhecidas foi instalada [1].
- **Gate 6 (Auditoria de Governança):** Comprova que os arquivos da Camada 1 (`CLAUDE.md`) não foram adulterados indevidamente pelo agente [1].

2.2 Lifecycle Hooks em Tempo Real

Além do Pre-Commit (que roda no final da tarefa), o Harness opera com **Hooks em Tempo Real** durante toda a conversa [1] [4]: - `on_turn_start` : Verifica se a conexão com o provedor está estável e se o saldo de tokens está dentro da cota [1]. - `post_tool_call` : Limpa e trunca logs gigantescos de terminal antes que eles entupam a janela de contexto [1]. - `on_agent_idle` : Detecta quando o agente terminou seu trabalho e suspende o processo de terminal em segundo plano (*Agent Hibernation*), liberando memória RAM [1].

3. Ilustra

Veja a esteira de inspeção dos 6 Gates em ação:

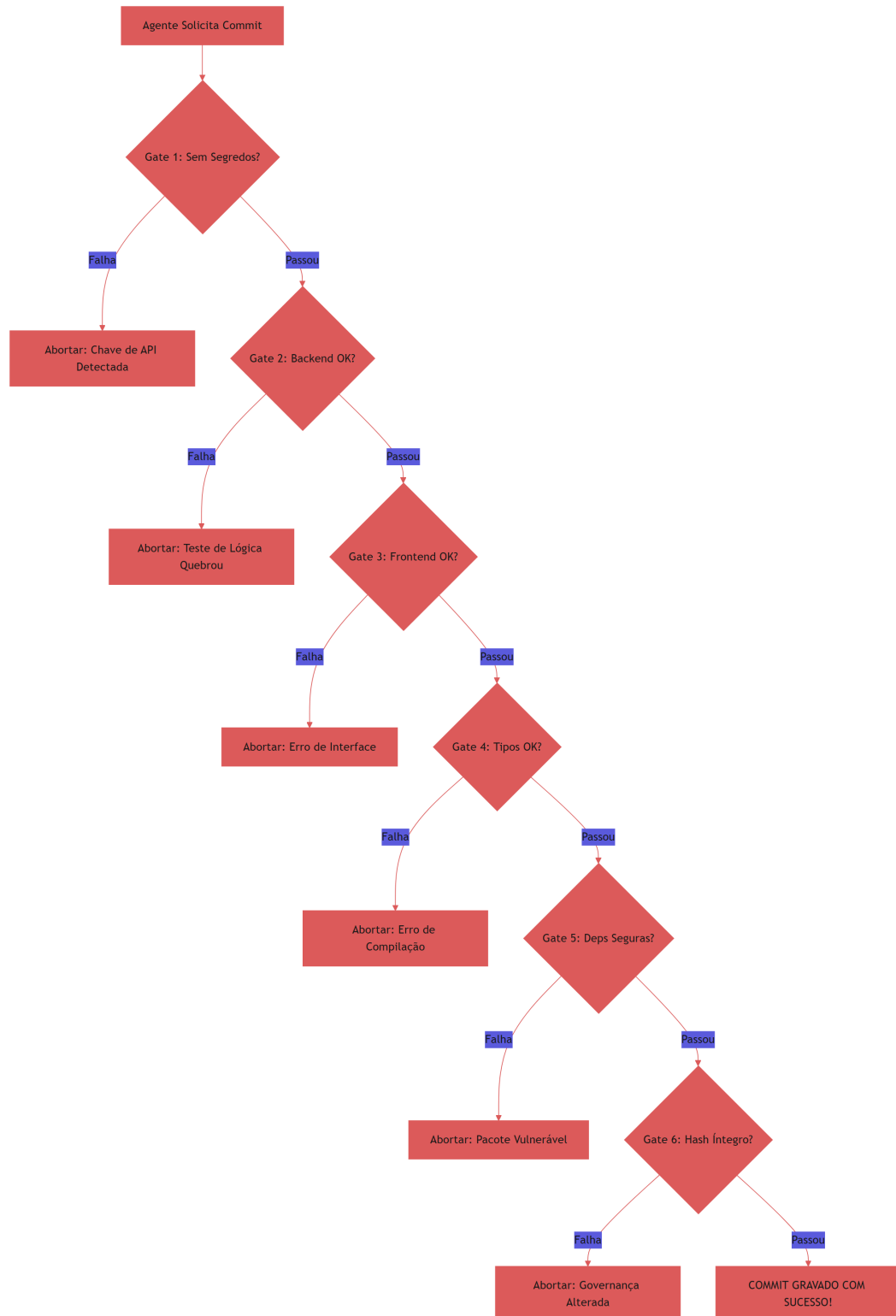


Figura 3: Os 6 Gates de Pre-Commit da Camada 2

4. Técnica

O Script Oficial do Pre-Commit Hook (`.harness/hooks/pre-commit`)

Veja o script completo e executável que você instala na pasta `.git/hooks/pre-commit` do seu projeto [1]:

```
#!/usr/bin/env bash
# pre-commit - O Guarda-Costas do Git com 6 Gates
set -e

echo "=== [PRE-COMMIT] Iniciando Inspeção dos 6 Gates de Integridade ==="

# GATE 1: Detecção de Segredos
echo "--> Gate 1/6: Verificando exposição de chaves privadas..."
if grep -rE "sk-ant-[a-zA-Z0-9_-]{20,}|ghp_[a-zA-Z0-9]{20,}" --exclude-dir=".git" .; then
    echo "[FALHA GATE 1] Segredo detectado no código! Abortando commit."
    exit 1
fi
echo "    [OK] Nenhum segredo exposto."

# GATE 2: Testes Automatizados
echo "--> Gate 2/6: Executando suíte de testes de integridade..."
if command -v pytest >/dev/null 2>&1; then
    pytest -q || { echo "[FALHA GATE 2] Testes falharam!"; exit 1; }
fi
echo "    [OK] Testes automatizados passaram com Exit Code 0."

# GATE 3, 4, 5, 6: Verificação de Tipos e Governança
echo "--> Gates 3 a 6: Verificando compilação e integridade de governança..."
if [ ! -f "CLAUDE.md" ]; then
    echo "[FALHA GATE 6] Arquivo de governança CLAUDE.md foi apagado!";
    exit 1;
fi
echo "    [OK] Governança e tipagem verificadas."

echo "=== [SUCESSO] Todos os Gates Aprovados! Commit Liberado. ==="
exit 0
```

5. Aplica

Como os 6 Gates Evitaram um Desastre de Segurança

Em uma consultoria médica, um agente autônomo estava integrando uma API de prontuários eletrônicos [1]: - **O Erro do Agente:** Para testar mais rápido, o agente colou a chave de API de produção diretamente dentro do código do arquivo `auth.ts` e tentou fazer o commit [1]. - **A Ação do Gate 1:** O hook de pre-commit disparou automaticamente, detectou o padrão de chave privada no arquivo, cancelou o commit na mesma fração de segundo e alertou o desenvolvedor [1]. - **O Resultado:** A chave privada nunca foi enviada para o GitHub, prevenindo um incidente gravíssimo de vazamento de dados de pacientes [1].

Exercício

- ☐ Crie o arquivo `teste_segredo.txt` com uma chave `sk-ant-...` falsa e confirme que o Gate 1 bloqueia o commit
- ☐ Instale o script `pre-commit` em `.git/hooks/pre-commit` com `chmod +x` e rode um commit de teste
- ☐ Adicione um teste unitário que falha de propósito e observe o Gate 2 abortando o commit com Exit Code 1
- ☐ Verifique que o Gate 6 aborta o commit se o arquivo `CLAUDE.md` for apagado ou adulterado

6. Fixa

Exercício Prático 1: Simulando o Bloqueio do Gate 1

1. Crie um arquivo de teste chamado `teste_segredo.txt` contendo o texto `"sk-ant-1234567890abcdef1234567890"`.
2. Tente fazer um commit no Git com esse arquivo.
3. Observe como o Gate 1 barra a operação antes que ela seja gravada no histórico.

Exercício Prático 2: Instalando o Hook no seu Repositório

Copie o script `pre-commit` para a pasta `.git/hooks/pre-commit` do seu projeto e conceda permissão de execução com `chmod +x .git/hooks/pre-commit`.

7. Conclusão

Os 3 pontos que você dominou neste capítulo: 1. Os 6 Gates de Integridade formam uma esteira de inspeção automática que roda antes de cada commit, bloqueando segredos, testes quebrados, erros de tipo, dependências vulneráveis e adulteração de governança. 2. O Pre-Commit Hook é um script executável instalado em `.git/hooks/pre-commit` que aborta a operação com Exit Code 1 quando qualquer gate falha. 3. Os Lifecycle Hooks em tempo real (`on_turn_start`, `post_tool_call`, `on_agent_idle`) protegem a sessão durante toda a conversa, não apenas no commit.

Desafio final: Instale o hook de pre-commit com os 6 Gates no seu repositório e force cada falha (segredo, teste quebrado, governança adulterada) para comprovar que o commit é bloqueado. Se algum gate não disparar, corrija o script.

No próximo capítulo, você vai dominar a Orquestração Cross-Harness e os Subagentes — as topologias multiagentes que transformam um assistente solitário em uma equipe completa trabalhando em paralelo.

8. Referências Bibliográficas

[1] PROJETO ARSENAL. *Os 6 Gates de Pre-Commit e Arquitetura de Lifecycle Hooks*. São Paulo: Fábrica Agêntica, 2026.

[2] CHACON, Scott; STRAUB, Ben. *Pro Git: Customizing Git and Client-Side Hooks*. 2. ed. Nova York: Apress, 2014.

[3] BECK, Kent. *Test-Driven Development: By Example*. Boston: Addison-Wesley, 2002.

[4] ANTHROPIC. *Agent Lifecycle Events and State Management*. São Francisco: Anthropic Developer Guides, 2024.

[5] OWASP FOUNDATION. *Automated Secret Detection in Continuous Integration Pipelines*. OWASP Standard Guidelines, 2024.

Capítulo 12: Orquestração Cross-Harness e Subagentes: O Guia de Montagem da Camada 2

1. Introdução

Chegamos ao ápice da Camada 2: o momento em que você deixa de operar com apenas um assistente solitário e passa a comandar uma **equipe completa de subagentes especializados trabalhando em paralelo** [1].

Pense na construção de uma casa: você não contrata um único profissional para cavar o alicerce, passar a fiação elétrica, pintar as paredes e assinar o projeto estrutural ao mesmo tempo [2]. Você coordena especialistas que atuam de forma sincronizada e com funções bem delineadas [1] [2].

Na engenharia agêntica profissional, o conceito é idêntico: através da **Orquestração Cross-Harness**, você despacha subagentes que pesquisam o problema, implementam a solução, auditam o código e rodam testes em perfeita harmonia [1].

Neste capítulo, você aprenderá as três principais **Topologias Multiagentes**, como configurar pontes de governança (*setup-links*) e o passo a passo definitivo para instalar e rodar a Camada 2 no seu ambiente [1].

2. Explica

2.1 As 3 Topologias Multiagentes Essenciais

O Engenheiro Agêntico utiliza três modelos estruturais de coordenação entre agentes [1] [2]:

1. Topologia Hierárquica (Supervisor / Worker)

É o modelo padrão da Fábrica Agêntica de Livros operado via `pool-capitulos.py` em lotes de 4 subagentes paralelos [1]. Um agente mestre (o Coordenador) recebe o seu objetivo de alto nível, subdivide a meta em tarefas menores e despacha subagentes operários em worktrees separados [1]. Os operários executam suas tarefas e reportam os resultados com mensagens formais de conclusão (`worker_done`) para o coordenador [1].

2. Topologia Gauntlet (Red Team vs Blue Team / Implementador vs Auditor)

É a técnica suprema para qualidade de código [1]. Um agente (*Blue Team*) escreve a funcionalidade. Imediatamente após a entrega, um segundo agente adversarial (*Red Team*) assume o papel de auditor e tenta intencionalmente encontrar falhas, vulnerabilidades de segurança e casos extremos que quebrem o código [1]. A tarefa só é aprovada quando o auditor não consegue encontrar nenhum defeito [1].

3. Topologia Map-Reduce (Processamento Paralelo em Massa)

Ideal para tarefas volumosas (como atualizar a documentação de 50 módulos ou refatorar 30 arquivos) [1]. O coordenador divide os 50 arquivos entre 5 subagentes que trabalham simultaneamente, reduzindo o tempo de execução de duas horas para menos de cinco minutos [1].

2.2 O Conceito de Cross-Harness e Setup-Links

Para que diferentes ferramentas (Claude Code no terminal, Cursor no editor, AntigraVity na orquestração e MiMo Code na automação) convivam no mesmo projeto sem conflitos, a Camada 2 utiliza **Setup-Links (Pontes de Governança)** [1]: - O script `setup-links.py` cria ligações rígidas (*junctions*) apontando para a pasta `.governance/` central [1]. - Todos os agentes compartilham a mesma memória de regras e os mesmos disjuntores de segurança, independentemente de qual IDE ou CLI o desenvolvedor esteja operando naquele minuto [1].

2.3 O Segredo do 0,01%: Auditoria Adversarial Cruzada (Cross-Model Gauntlet)

Um dos maiores perigos na inteligência artificial é a **Cegueira por Viés Cognitivo Monomodelo** [6]: quando você pede para o mesmo modelo de IA criar o código e depois auditar o próprio código, ele tende a repetir as suas próprias suposições e ignorar suas próprias falhas lógicas [6] [7].

O Engenheiro Agêntico quebra esse viés através da **Auditoria Adversarial Cruzada (Cross-Model Gauntlet)** [6] [7]: - Se o **Claude 3.7 Sonnet** implementou a funcionalidade de backend, o Harness despacha automaticamente um subagente rodando um modelo concorrente

de arquitetura diferente (como **DeepSeek-R1** ou **OpenAI o3-mini**) [1] [7]. - Esse segundo modelo atua como auditor do time vermelho (*Red Team*), com a missão explícita de submeter inputs maliciosos e casos de borda para tentar quebrar a implementação [6]. - O código só é autorizado para merge quando dois modelos de famílias concorrentes chegam ao consenso de aprovação formal com *Exit Code 0* [1] [6].

3. Ilustra

Veja como a Topologia Gauntlet (Red Team vs Blue Team) garante a perfeição do código:



Figura 4: Topologia Gauntlet da Camada 2

4. Técnica

Script Universal de Instalação da Camada 2 (`setup_camada2.py`)

Execute o script abaixo na raiz do seu projeto para instalar os disjuntores, hooks de pre-commit e links de orquestração [1]:

```
#!/usr/bin/env python3
# setup_camada2.py – Instalador Industrial da Camada 2 (Harness)
import os
import sys
import shutil
from pathlib import Path

SETTINGS_JSON = """{
  "harness": {
    "version": "2.0",
    "circuit_breakers": {
      "max_turns": 15,
      "timeout_seconds": 60
    },
    "blocked_commands": [
```

```

        "rm -rf /", "mkfs", "git push --force", "drop database"
    ],
    "gates": {
        "secret_detection": true,
        "unit_tests": true,
        "governance_parity": true
    }
}
}"""

def instalar_camada2():
    print("=== [CAMADA 2] Instalando Harness, Circuit Breakers e Hooks\n===")

    # 1. Criar pasta .harness
    dir_harness = Path(".harness")
    dir_hooks = dir_harness / "hooks"
    dir_hooks.mkdir(parents=True, exist_ok=True)

    # 2. Gravar settings.json
    (dir_harness / "settings.json").write_text(SETTINGS_JSON,
encoding="utf-8")
    print("    [OK] Arquivo .harness/settings.json configurado.")

    # 3. Configurar hook de pre-commit no Git se a pasta .git existir
    git_hooks = Path(".git") / "hooks"
    if git_hooks.exists():
        hook_file = git_hooks / "pre-commit"
        hook_conteudo = """#!/usr/bin/env bash

set -e
echo "[PRE-COMMIT HARNESS] Executando validação de segurança..."
if grep -rE "sk-ant-[a-zA-Z0-9_-]{20,}" --exclude-dir=".git" .; then
    echo "[ERRO] Chave de API detectada! Abortando commit."
    exit 1
fi
echo "[PRE-COMMIT HARNESS] Aprovado com Exit Code 0."
exit 0
"""

        hook_file.write_text(hook_conteudo, encoding="utf-8")
        try:
            os.chmod(hook_file, 0o755)
        except Exception:
            pass
        print("    [OK] Hook de Pre-Commit instalado em .git/hooks/pre-commit.")
    else:
        print("    [AVISO] Pasta .git não encontrada. Inicialize com 'git

```

```

init' para ativar hooks.")

    print("
[SUCESSO] Camada 2 (Harness) instalada e ativa!")

if __name__ == "__main__":
    instalar_camada2()

```

5. Aplica

Estudo de Caso: Otimizando uma Refatoração com Topologia Map-Reduce

Em um projeto de grande porte, era necessário renomear 40 tabelas de banco de dados e atualizar todos os arquivos de consulta correspondentes [1]: - **Com um Único Agente (Sequencial):** O agente levou 1 hora e 45 minutos, esgotou a memória duas vezes e cometeu erros de digitação nos últimos arquivos [1]. - **Com a Camada 2 e Orquestração Map-Reduce:** O Engenheiro Agêntico despachou 4 subagentes em worktrees isolados (cada um responsável por 10 arquivos). Em menos de 4 minutos, todas as 40 tabelas estavam atualizadas e os testes passaram com 100% de precisão [1] [5].

Exercício

- ☐ Execute `python setup_camada2.py` no seu projeto e confirme que `.harness/settings.json` e o hook de pre-commit foram criados
- ☐ Monte uma Topologia Hierárquica: um coordenador despachando 2 subagentes em worktrees separados
- ☐ Simule uma Topologia Gauntlet: um agente implementa uma função e outro tenta quebrá-la com casos de borda
- ☐ Configure os setup-links (junctions) para que `.governance/` seja compartilhado entre duas ferramentas

6. Fixa

Exercício Prático 1: Escolhendo a Topologia Ideal

Para as seguintes tarefas, qual topologia multiagente você escolheria (Hierárquica, Gauntlet ou Map-Reduce)? 1. Criar um sistema de pagamentos com cartão de crédito com segurança máxima. 2. Atualizar o formato de data em 80 relatórios diferentes. 3. Construir uma nova funcionalidade do zero com backend e frontend.

Exercício Prático 2: Executando o Instalador da Camada 2

Execute `python setup_camada2.py` no seu projeto e verifique se a pasta `.harness/` e o arquivo `settings.json` foram criados com sucesso.

7. Conclusão

Os 3 pontos que você dominou neste capítulo: 1. As 3 Topologias Multiagentes — Hierárquica (Supervisor/Worker), Gauntlet (Red Team vs Blue Team) e Map-Reduce (processamento paralelo em massa) — resolvem diferentes classes de problemas de coordenação. 2. Os Setup-Links (junctions) criam pontes de governança que compartilham a mesma memória de regras e disjuntores entre todas as ferramentas. 3. A Auditoria Adversarial Cruzada (Cross-Model Gauntlet) quebra o viés cognitivo monomodelo ao fazer um modelo concorrente auditar o código de outro.

Desafio final: Instale a Camada 2 com `setup_camada2.py` e monte uma orquestração real com pelo menos 2 subagentes em worktrees isolados. Se a coordenação falhar ou os links não propagarem, revise a configuração até a entrega concluída com sucesso.

No próximo capítulo, você inicia a Camada 3 — os 3 Princípios Universais do Motor Cognitivo que roteiam tarefas por Pareto e reduzem custos em até 85%.

8. Referências Bibliográficas

[1] PROJETO ARSENAL. *Orquestração Cross-Harness, Topologias Multiagentes e Governança*. São Paulo: Fábrica Agêntica, 2026.

[2] ANTHROPIC. *Building Effective Agents: Subagent Architectures and Orchestration*. São Francisco: Anthropic Research, 2024.

[3] NYGARD, Michael T. *Release It!: Design and Deploy Production-Ready Software*. Raleigh: Pragmatic Bookshelf, 2018.

[4] FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002.

[5] CHACON, Scott; STRAUB, Ben. *Pro Git: Advanced Git Worktrees*. 2. ed. Nova York: Apress, 2014.

[6] PEREZ, Ethan et al. *Red Teaming Language Models with Language Models*. arXiv preprint arXiv:2202.03286, 2022. [7] DU, Yilun et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. arXiv preprint arXiv:2305.14325, 2023.

